

● About this Software: :

This software provides a comprehensive solution for managing the localization environment of Marvel Heroes Omega (MHO), including: adding new languagePack, converting game StringFiles to text files, and rebuilding text files back into StringFiles.

The primary purpose of this software is to facilitate the Traditional Chinese localization of Marvel Heroes Omega. Its core design focuses on the efficient translation of entire language StringFiles.

(Note: The "Add New LanguagePack" feature clones one of the six built-in languages; the actual content must still be translated manually.)

- Developed in Python with assistance from ChatGPT.
- Free to distribute and use. This software has not undergone extensive testing; compatibility and reliability are not guaranteed.
- Tested only on Windows 10 x64 (Chinese Version)—successfully completed the Chinese localization of the game.
- All English content was translated using ChatGPT and Gemini.
- Designed for game version 1.52; compatibility with other versions has not been tested.
- Prototype demonstration video for the Traditional Chinese version: <https://youtu.be/x3FhU4aS6Zg>
- MHO localization information is sourced from the MHServerEMU project. Special thanks to all project members.
<https://github.com/Crypto137/MHServerEmu/blob/master/docs/GameData/Locale.md>
<https://discord.com/invite/hjR8Bj52t3>

Contents : Basic knowledge of the MHO locale system 0. Getting Started 1.Clone a new LanguagePack 2. Exporting StringFiles to JSON 3. Rebuild StringFiles from JSON 4.Convert old JSON to new JSON 5. About Tokens: \$, #, and #/ 6. About generated files	F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ==== MarvelHeroes Omega 語系大師 v1.0 2026.05.01 ==== 1. 克隆一個新的語言包 2. 字串檔轉出 JSON 文字檔 3. JSON 文字檔重建字串檔 4. 舊版 JSON 轉為新版 JSON 5. 結束 請選擇功能(1~5): F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ==== MarvelHeroes Omega LocaleMaster v1.0 2026.05.01 ==== 1. Clone a new languagePack 2. Convert Strings to JSON 3. Rebuild Strings from JSON 4. Migrate Legacy string JSON 5. Exit Please select a function (1~5): _
---	---

• . Basic knowledge of the MHO locale system

MHO includes 6 built-in languages : English/French/German/Portuguese/Russian/Spanish; plus 3 planned but unused languages : Chinese/Japanese/Korean; and 3 unknown languages (sg1/sg2/sg3).

Each MHO LanguagePack consists of 1 language configuration file, 1 StringFiles folder, and 4 StringFiles.

The path for the LanguagePack is shown in the image below, located at \GameFolder\Data\Game\Loco\ :

As shown on the right, \Marvel Heroes Omega.2.16a\ is the {GameFolder}.

The game folder name can be changed freely and may differ for each user, so it is referred to as {GameFolder}.

eng.all.locale is the English LanguagePack configuration file.

eng.all is the string folder for the English LanguagePack.

Note: the English LanguagePack name is "eng.all", not "eng"; the same applies to other LanguagePacks.

The right image shows the contents of the \eng.all\ folder.

It contains 4 string files named eng.all_*.string.

StringFiles are the MHO game message files; most in-game text is stored here.

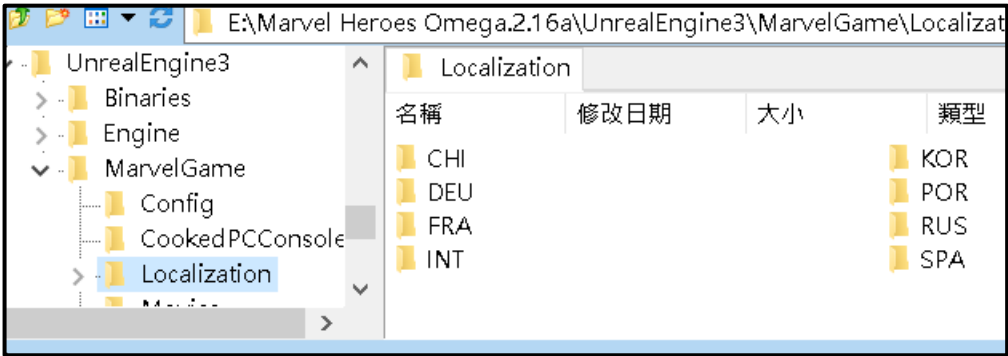
Text not included in these files is usually provided by the server side, such as achievements and leaderboards.

In addition to LanguagePacks, the game cutscenes also support multi-language subtitles, including Chinese and Korean (no Japanese subtitles).

Previously, Chinese/Korean subtitles could not be used due to the absence of corresponding locale support; Now, they can be viewed properly.

The subtitle file path is shown below :

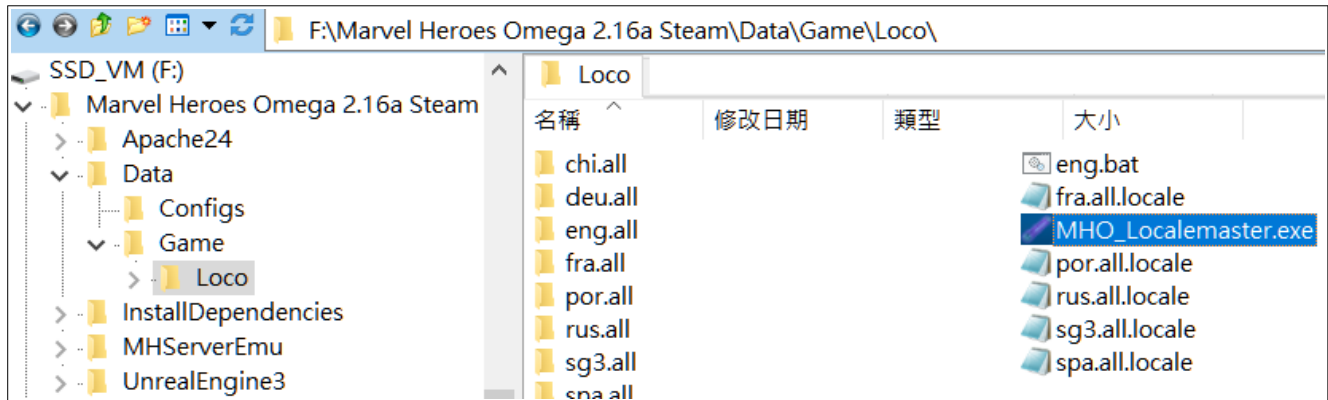
.\{GameFolder}\UnrealEngine3\MarvelGame\Localization\



● 0. Getting Started

- This tool consists of a single file, [MHO_Localemaster.exe](#). It must be placed in `.\{GameFolder}\Data\Game\Loco\` before use, as shown below:

(It must be the \Loco\ directory from a complete game installation, not a copied standalone \Loco\ folder, as it checks the game executable version...)



- The program supports Traditional Chinese and English interfaces. It automatically selects based on the Windows language version, or you can specify it with a parameter : `MHO_Localemaster.exe <arg> ; 0 = English, 1 = Chinese.`
- Close the game before using this tool.
- When loading string JSON files, it automatically checks whether the JSON format is valid and reports any errors. Each run reports only the first error found.
- Before performing any destructive operation, the program automatically creates a backup.

- This program has two main use cases:

1. Modify an existing Languages:

For any of the 9 supported languages (English, French, German, Portuguese, Russian, Spanish, Chinese, Japanese, Korean), export the target Language into JSON text files, edit the text, then rebuild it back into StringFiles.

2. Create a new Language:

In the narrow sense, this refers to Chinese, Japanese, or Korean. In a broader sense, it can be any language, as long as MarvelFonts_SF.upk can render the target characters.

This process involves creating a .locale file, clone the LanguagePack folder and StringFiles, generate JSON text files, translate them, and rebuild the StringFiles.

● 1. Clone a new LanguagePack :

Clone an existing locale using any of the Chinese/Japanese/Korean LanguagePacks.

The “new [LanguagePack] file name” in the tool is fixed as chi.all, jpn.all, or kor.all, simply one of these three options used as the folder name.

However, neither the [New LanguagePack] name nor the [New Language] name determines the actual language content. The real language content is defined by the StringMap JSON files generated in [Function 2].

For example, the diagram below creates a [New Language] named “japanese” using the jpn.all folder. The string JSON generated in [Function 2] is translated into Chinese, then rebuilt into jpn.all StringFiles via [Function 3]. As a result, selecting this “japanese” locale in-game will display Chinese text (confirmed behavior).

Steps : 1. Select Sample language (1 of 6). 2. Select new LanguagePack name (1 of 3). 3. Enter new Language name, this will be displayed in the in-game language menu.	F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ---- MHO Clone a new language pack (for translating into a new language) ---- 1. Select [Sample language]: 1.Eng 2.Fra 3.Deu 4.Por 5.Rus 6.Spa (1~9)1 2. Select [New language pack] file name: 7.Chi 8.Jpn 9.Kor (7~9)8 3. Enter [New language] name (max 16 bytes): Japanese > Sample language: eng.all, New language pack: jpn.all, New language name: Japanese Confirm? (Y/N/B/Q):
Result : - Generates 5 files and 1 folder. - If files/folders already exist, they will be backed up first.	F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe 2. Select [New language pack] file name: 7.Chi 8.Jpn 9.Kor (7~9)8 3. Enter [New language] name (max 16 bytes): Japanese > Sample language: eng.all, New language pack: jpn.all, New language name: Japanese Confirm? (Y/N/B/Q): y > Generating new language pack: - jpn.all\jpn.all_3FFFFFFFFFFFFFFF.string - jpn.all\jpn.all_7FFFFFFFFFFFFFFF.string - jpn.all\jpn.all_BFFFFFFFFFFFFFFF.string - jpn.all\jpn.all_FFFFFFFFFFFFFFFF.string - jpn.all.locale (Surprise! The new language pack is just an empty shell. It's still eng.all inside, so it's translation time.) --- Press any key to continue ---
Game example : - 9 languages available simultaneously.	Language 中文(Taiwan) 中文(Taiwan) Deutsch English Français 日本語 Language 中文(Taiwan) 日本語 한국어(Korean) Português Русский Español

• 2. Exporting StringFiles to JSON

This function converts the target language StringFiles into text files and generates 5 files:
MainMap.json , SrcSampTok.json , and StringMap.json are the primary files; only StringMap.json is editable.

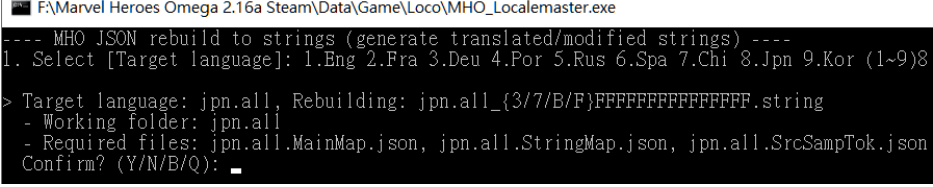
dbg-locked_tokens.json and dbg-src_unpaired_tags.json are reference/debug files from the sample language,
These are not required for the process and are for reference only.
(See • 6. for details of each file.)

Steps : 1. Select sample language. it must be an existing language. 2. Select output language if it does not exist, the LanguagePack folder will be created automatically (without the .locale file). - Unless [sample language] & [output language] are the same (editing the original language), it is not recommended to use the built-in six languages as the output Language.	<pre> F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ---- MHO Strings export to JSON (for translation/editing string content) ---- 1. Select [sample language]: 1.Eng 2.Fra 3.Deu 4.Por 5.Rus 6.Spa 7.Chi 8.Jpn 9.Kor (1~9)1 2. Select [output language]: 1.Eng 2.Fra 3.Deu 4.Por 5.Rus 6.Spa 7.Chi 8.Jpn 9.Kor (1~9)8 > sample language: eng.all, output language: jpn.all Confirm? (Y/N/B/Q): _ </pre>
Result : - After confirmation, 5 JSON files are generated in the [output language] folder. - This function automatically backs up the languagePack folder.	<pre> > Parsing started, generating files: Output folder: jpn.all - jpn.all.SrcSampTok.json (Calibration sample, do not modify) - jpn.all.dbg-locked_tokens.json (Locked string list, reference only) - jpn.all.dbg-src_unpaired_tags.json (Unmatched text tag list, reference only) - jpn.all.MainMap.json (StringMap main table, do not modify) - jpn.all.StringMap.json (StringMap text content, editable string content) --- Press any key to continue --- _ </pre>
Example : Contents of the jpn.all.StringMap.json game message text file. - creation_time must not be modified, it serves as a verification code to ensure synchronization with the other two JSON files. - Only the text inside "string": "..." should be edited (highlighted area). - The status field can only modify lock and no_audit (0 or 1). dollar/hash/hash_slash are reference-only, and have no effect if changed - dollar/hash/hash_slash indicate how many \$variables, #tags, and #/tags exist in the string.	<pre> { "creation_time": "2026-05-06 01:27:20", "entries": [{ "seq_no": 1, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "" }, { "seq_no": 2, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "Bulletproof. Figures...\n—Punisher" }, { "seq_no": 3, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "Mission Failure" }, { "seq_no": 4, "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", "string": "HYDRA Excavation" }] } </pre> - lock = 1, means the string is locked, and will not use this value; the sample language string will be kept instead. - no_audit = 1, disables validation of [\$, #, #/] counts against the sample language; no log will be generated.

● 3. Rebuild StringFiles from JSON

Rebuild the 3 main JSON files into 4 StringFiles. Existing StringFiles will be overwritten without backup.

The target languagePack folder must contain the corresponding set of [MainMap.json](#), [SrcSampTok.json](#), and [StringMap.json](#) files generated by [Feature 2].

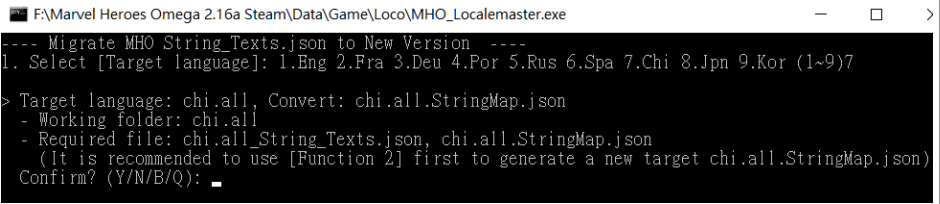
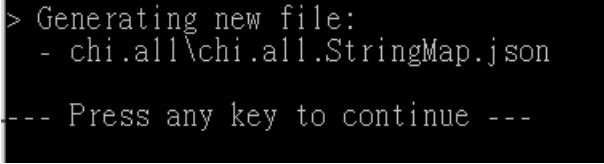
Steps : 1. Select target language. 2. After confirmation, generate the target language StringMap files directly.	 <pre> F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHO_Localemaster.exe ---- MHO JSON rebuild to strings (generate translated/modified strings) ---- 1. Select [Target language]: 1.Eng 2.Fra 3.Deu 4.Por 5.Rus 6.Spa 7.Chi 8.Jpn 9.Kor (1~9)8 > Target language: jpn.all, Rebuilding: jpn.all_{3/7/B/F)FFFFFFFFFFFFFFF.string - Working folder: jpn.all - Required files: jpn.all.MainMap.json, jpn.all.StringMap.json, jpn.all.SrcSampTok.json Confirm? (Y/N/B/Q): _ </pre>
Result : - In addition to the four StringFiles, a .Log file will also be generated. - The log records strings with mismatched counts of \$, #, and #/ tokens. - Each execution of [Function 3] will overwrite the .Log file.	<pre> > Checking JSON consistency: - jpn.all.MainMap.json Created: 2026-05-08 15:28:38 - jpn.all.StringMap.json Created: 2026-05-08 15:28:38 - jpn.all.SrcSampTok.json Created: 2026-05-08 15:28:38 > Generating new Strings: - jpn.all\jpn.all_3FFFFFFFFFFFFFFF.string - jpn.all\jpn.all_7FFFFFFFFFFFFFFF.string - jpn.all\jpn.all_BFFFFFFFFFFFFFFF.string - jpn.all\jpn.all_FFFFFFFFFFFFFFFF.string - jpn.all.dbg-audit_tokens.log (String [\$vars, #tags] audit log) --- Press any key to continue --- _ </pre>
Example : - The image on the right shows the contents of the Log file. - It lists the seq_no of problematic strings, along with the OrigString and DestString for comparison and verification. - MissTokens refer to mismatches in the counts of \$, #, and #/ tokens. As shown on the right : #mission#: 9/8 (#/mission#: 9/8) means #mission# appears 8 times in the OrigString but 9 times in the DestString. This indicates either an editing error or an intentional change. If it is intentional, you can set no_audit = 1 to skip validation for this entry.	<pre> Lists entries in StringMap.json where [\$ variables] or [# , #/ tags] do not ----- Sequence= "seq_no":1086 OrigString= Use any waypoint or terminal to travel to #mission#Midtown#/mis DestString= 使用任何路標或終端機傳送到#mission#中城#/mission#或是#mission# MissTokens= #mission#: 9/8, #/mission#: 9/8 ----- Sequence= "seq_no":1347 OrigString= #waypointheader#\$RegionName\$#/waypointheader# #waypointlabel DestString= #waypointheader#\$RegionName\$#/waypointheader# #waypointlabel MissTokens= #/waypointlabel#: 2/1 ----- Sequence= "seq_no":1629 OrigString= #waypointheader#Airfield Showdown (Cosmic)#/waypointheader# DestString= #waypointheader#機場決戰 (宇宙)#/waypointheader# #waypointla MissTokens= #/waypointlabel#: 2/1 ----- </pre>

● 4. Convert old JSON to new JSON

This program is a rewrite of a previous version. This feature allows you to convert old JSON files into the new JSON format.

Prerequisites:

- Old format files : [???all_String_Terms.json](#), [???all_String_Texts.json](#).
- New format files : A set of new JSON files where both the [Source/Target] languages match the old files.
- Place the two old JSONs and the three new JSONs into the target LanguagePack folder.

Steps : 1. Select target language. 2. After confirmation, generate the new JSON files.	 <pre>F:\Marvel Heroes Omega 2.16a Steam\Data\Game\Loco\MHQ_Localemaster.exe ---- Migrate MHQ String_Texts.json to New Version ---- 1. Select [Target language]: 1.Eng 2.Fra 3.Deu 4.Por 5.Rus 6.Spa 7.Chi 8.Jpn 9.Kor (1~9)7 > Target language: chi.all, Convert: chi.all.StringMap.json - Working folder: chi.all - Required file: chi.all_String_Texts.json, chi.all.StringMap.json (It is recommended to use [Function 2] first to generate a new target chi.all.StringMap.json) Confirm? (Y/N/B/Q):</pre>
Result : This function migrates strings from the old format files into the new format files, which is why a pre-prepared set of new files is required.	 <pre>> Generating new file: - chi.all\chi.all.StringMap.json ---- Press any key to continue ----</pre>

● 5. About Tokens: \$, #, and #/

As shown in the image on the right, "Unlocks at Level: 32" is actually represented in the string file as:

"Unlocks at Level: \$unlocklevel\$", 32 is a variable value. Variables are enclosed by two \$ symbols and must not be modified or translated.

Special attention should be paid to the variable name inside the \$ symbols - unlocklevel. It usually maps to a separate standalone string "unlocklevel", which also must not be modified.

\$variable_name\$ is recorded in the log file as a "dollar (\$)".

Next, looking at "When you use a Serum Power:", in the StringMap it is actually:

```
"#emphasis#When you use a #powerkeyword#$procKeyword.param2.keyword$#/powerkeyword# Power:#/emphasis#"
```

Serum is the value of the variable \$procKeyword.param2.keyword\$.

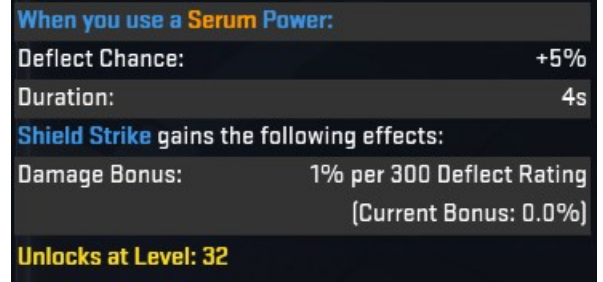
The text (Serum) enclosed by #powerkeyword# and #/powerkeyword# becomes bold orange text,

#format_tag_start# and #/format_tag_end# change the formatting of the text they enclose. A tag_start is wrapped with # # , while an tag_end is wrapped with #/ #. Normally, #tag_start# and #/tag_end# must appear as a pair, if they are not paired, it is a bug.

Like \$variable\$, #format_tags# and #/format_tags# must not be modified or translated,

The #tag_name# usually also has a corresponding standalone string entry: "tag_name", which likewise must not be modified.

Using the two examples above, Besides the tokens within the strings themselves : \$unlocklevel\$, #emphasis#, #/emphasis#, #powerkeyword#, #/powerkeyword# and \$procKeyword.param2.keyword\$, if you find the following Entry_Strings, they likewise must remain unchanged: "unlocklevel", "emphasis", "powerkeyword", "procKeyword.param2.keyword".



The image on the right shows the most obvious example of a #tag abnormality in the game:

The original English string is as follows :

```
"Use to claim a mystery reward! \n\n
#QualityEpic# Epics: #/QualityEpic#
\n*Gambit End of the World Costume (Exclusive)\n*Cyclops Noir
Costume (Exclusive)\n*Bamf Pet\n*Ultimate Upgrade Token\n\n
#RarityRare# Rares: #/RarityRare#
\n*X-Force Red Spotlight Visual Artifact (Exclusive) \n*X-Men Blue
Spotlight Visual Artifact (Exclusive) \n*Combo Boost\n*Infinity Orb
Chest\n\n
#rarityuncommon# Commons: #/rarityuncommon#
\n*S.H.I.E.L.D. Supply Boost\n*Experience Boost\n*Currency Boost"
```

What was wrong?

The standalone tag-name string "rarityuncommon" did not exist, only "RarityunCommon" was found.

After changing the tag name to "RarityunCommon", the issue was fixed. The Chinese version already includes this correction.



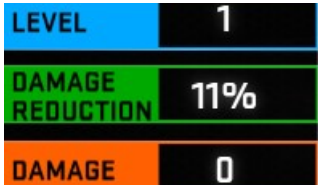
#tag_name_start# is recorded in the log file as a "hash (#)", while #/tag_name_end# is recorded as a "hash_slash (#/)"

• 6. About generated files

- Three most important StringMap JSON files:

MainMap.json :	Stores the primary data for the four string files; this file should not be modified.
StringMap.json :	Stores only the text portion of the string files, allowing translators to focus on the text while ignoring the data in MainMap.json.
SrcSampTok.json :	Stores the sample language string text, along with the counts of \$variables, #tags, and #/tags contained in each string.
- StringMap.json is the only editable file among all generated files; all others are data files that should not be modified.	
- Each entry in StringMap.json contains "status": "lock=0, no_audit=0, dollar=0, hash=0, hash_slash=0", Among these, lock and no_audit are editable parameters, while dollar, hash and hash_slash are reference-only values and are not read back during reconstruction.	
lock : Default is 0, If set to 1, the string is locked and cannot be modified, the original value from "SrcSampTok.json" will be forced.	
no_audit : Default is 0, If set to 1, the string will skip validation of "\$variables" and "#tags", and no log entry will be generated.	

- Two reference files for debugging :

dbg-locked_tokens.json : This file lists all variable_names and tag_names for reference. By default, this program sets all strings identical to these "names" to lock=1 to prevent accidental modification. However, some non-variable/tag strings may be incorrectly locked. These must be manually excluded, or you can disable this behavior entirely by replacing all lock=1 with lock=0. Using the example on the right, both "Damage" and "Level" appear twice. Which one is a token name, and which one is actual in-game text? You'll have to distinguish and exclude them yourself. (The text in the game screenshot on the right appears to be in all caps, but they are actually "Level" and "Damage". There are many such "text traps" in the game - you'll understand once you start translating :)	<pre>{ "_comment": "The file lists the default locked", "creation_time": "2026-05-08 22:21:54", "total": 658, "tokens": ["AreaName", "DISPLAYNAME", "Damage", "DamageOverTimeDuration", "LegendaryMissionsComplete", "Level", "LevelRange", </pre> 
dbg-src_unpaired_tags.json: This file lists strings in the sample language where #tag# and #/tag# are not properly paired - i.e., original tagging bugs. These should be fixed first. In the example of string 1347 on the right, there are 2 #waypointlabel# tags but only 1 #/waypointlabel#, It's definitely broken and needs a fix!	<pre>{ "_comment": "This file Lists unmatched text tags in the src", "creation_time": "2026-05-08 22:21:54", "total": 93, "entries": [{ "seq_no": 1347, "string": "#waypointheader##RegionName##/waypointheade", "hash": { "#waypointlabel#": 2 }, "hash_slash": { "#/waypointlabel#": 1 } }, { "seq_no": 1629, "string": "#waypointheader#Airfield Showdown (Cosmic)#", "hash": { "#waypointlabel#": 2 }, "hash_slash": { "#/waypointlabel#": 1 } }] }</pre>

- The error-correction log file generated during StringFiles reconstruction :

dbg-audit_tokens.log : This file records tokens in StringMap.json that do not match the quantities in the source language. A token mismatch is not necessarily a bad thing; for instance, if the source string contains an original developer BUG, correcting it will trigger a log entry. If the target string is confirmed to be correct, you can set no_audit = 1 to exclude this entry from further checks.
--